MIDDLESEX UNIVERSITY

The Burroughs, Hendon, London NW4 4BT

CST3133 Coursework

Applied Data Science and AI Workflow

Unified Report – Dataset 1 (Structured) & Dataset 2 (Text)

Group Members

Dumitru Nirca	M00888146
Sebastian Robert Danci	M00886707
Maciej Kacper Ciba	M00864763
Peter Nagy	M00871555

Module	CST3133 – Advanced Topics in Data Science and AI
Module Leader	Dr. Halil Yetgin
Word Count	5,300 (approx., max 7000)

Academic Year 2024/2025

Contents

List of Figures

List of Tables

Section 1 – Traditional Machine Learning Workflow and Ethics (Dataset 1)

This section covers Dataset 1: the CSGO match prediction structured dataset.

Introduction and Objectives. The objective of this project is to apply a complete traditional machine learning workflow to predict CSGO match outcomes from pre-match statistics. Esports analytics increasingly informs team strategy, talent scouting, and performance benchmarking; accurate pre-match predictions can support these applications. This report documents: (1) dataset selection and problem formulation; (2) preprocessing and handling of data quality issues; (3) exploratory analysis and bias identification; (4) model development, evaluation, and interpretation; and (5) ethical considerations and mitigation strategies. We aim to achieve predictive accuracy above random chance (50%) while maintaining interpretability and documenting limitations.

1.1 Dataset Selection and Problem Definition (5%)

1.1.1 Dataset Overview

The dataset selected for this coursework is the **CSGO (Counter-Strike: Global Offensive) Match Prediction** dataset, which contains historical competitive match data from professional CSGO esports competitions. The dataset provides comprehensive team-level and player-level statistics recorded prior to each match. It is publicly available on Kaggle: https://www.kaggle.com/datasets/gabrieltardochi/counter-strike-global-offensive-matches. The dataset file `csgo_games.csv` is uploaded alongside this report.

The dataset in its raw form contains **3,787 rows** and **170 features**, expanding to **178 features** following feature engineering (Section 1.2). The processed dataset, after preprocessing and cleaning, retains **3,763 valid match records**.

Features are organised into two broad categories. Table 1 provides a detailed breakdown.

Table 1: Exact feature inventory and descriptions.

Category	Features (exact names and meanings)
Team-level (raw)	`t1_world_rank`, `t2_world_rank` (world rankings 1–∞); `t1_h2h_win_perc`, `t2_h2h_win_perc` (head-to-head win %); `t1_points`, `t2_points` (removed—data leakage).
Player-level (per player)	For each of `t1_player1`–`t1_player5` and `t2_player1`–`t2_player5`: `rating` (HLTV), `impact`, `kdr`, `dmr`, `kpr`, `apr`, `dpr`, `spr`, `opk_ratio`, `opk_rating`, `wins_perc_after_fk`, `fk_perc_in_wins`, `multikill_perc`, `rating_at_least_one_perc`, `is_sniper`, `clutch_win_perc`.
Engineered	`t1_avg_rating`, `t2_avg_rating`; `t1_avg_impact`, `t2_avg_impact`; `t1_avg_kdr`, `t2_avg_kdr`; `rank_diff`; `h2h_advantage`.

In total: 4 team-level (2 removed), 160 player-level (16 per player $\times$ 10 players), and 8 engineered features. The processed model uses **172 features** (170 raw minus 3 leakage, plus 8 engineered minus 3 metadata columns).

Target variable: `winner` — a binary categorical variable encoding which team won the match (`t1` = Team 1, `t2` = Team 2). The class distribution is near-perfectly balanced: 51.05% Team 1 wins (1,921 matches) and 48.95% Team 2 wins (1,842 matches), so no oversampling or class weighting is required.

1.1.2 Problem Definition

This project addresses a **binary classification problem**: predicting which team will win a CSGO match given only pre-match available statistics.

Specific business and analytical use cases:

- **Team management & scouting:** Front offices can use predictions to identify undervalued opponents or overvalued matchups before transfer windows, prioritise which matches require deeper opponent analysis, and benchmark expected win probability against actual results to evaluate coaching impact.
- **Tournament organisers:** Event producers can inform seeding decisions, bracket design, and scheduling (e.g., placing higher-stakes matches in prime-time slots) using pre-match win probability estimates.
- **Content & broadcast:** Analysts and broadcasters can generate pre-match story-

lines, highlight competitive gaps or upsets-in-the-making, and tailor commentary to the predicted closeness of a match.

- **Sponsorship & investment:** Brands and investors can assess team strength objectively when evaluating sponsorship ROI or portfolio performance, supplementing qualitative factors with quantitative win expectations.

The model output is a probability estimate, not a deterministic forecast; inherent match variance means even strong favourites can lose. Interpretability and documented limitations are therefore essential for responsible deployment.

1.2 Data Preprocessing (10%)

1.2.1 Handling Missing Values and Noise

In accordance with the coursework requirement to introduce NaN values to the dataset if they do not exist, the original dataset was modified: it already contained naturally occurring missing values (6,743 NaNs across 34 player-level columns), and additional NaN values were introduced in selected columns to create a noisy version for preprocessing. The focus of this section is on **identifying and mitigating** these data quality issues, not on the method of introduction.

The modified dataset with introduced and naturally occurring NaN values was then cleaned using the following imputation strategy:

- **Numerical columns:** Median imputation via scikit-learn's `SimpleImputer`. The median was preferred over the mean because it is robust to outliers — in player statistics, exceptional match performances can strongly skew arithmetic means.
- **Categorical columns:** Mode imputation, replacing missing values with the most frequently occurring category.

Before vs. after preprocessing (data quality comparison). Table 2 summarises the dataset state before and after preprocessing. The "before" state reflects the modified dataset with naturally occurring and introduced NaN values; the "after" state reflects the cleaned, imputed dataset ready for modelling.

Table 2: Data quality: before and after preprocessing.

Aspect	**Before (with NaN / noise)**	**After (processed)**
Rows	3,787	3,763
Columns / features	170	172 (numerical only)
Missing values	6,743+ (natural + introduced; imputed)	0
Outliers	Present (median imputation)	Retained, scaled
Leakage features	`t1_points`, `t2_points`, `points_diff`	Removed
Train / test split	–	3,010 / 753

Outlier and error detection methodology. Outliers were identified via: (1) **boxplot inspection** of key features (e.g., `rank_diff`, ratings) to detect extreme values; (2) **right-skewed distributions** in world rankings indicating high-end extremes; and (3) **domain knowledge**—exceptional player performances (e.g., KDR > 2) are valid but rare. Errors (e.g., impossible values such as negative percentages) were checked via `df.describe()` and range validation; none were found. Median imputation was chosen specifically because it is **robust to outliers**: unlike the mean, the median is not distorted by extreme values, making it appropriate for player statistics where single extraordinary performances can skew distributions.

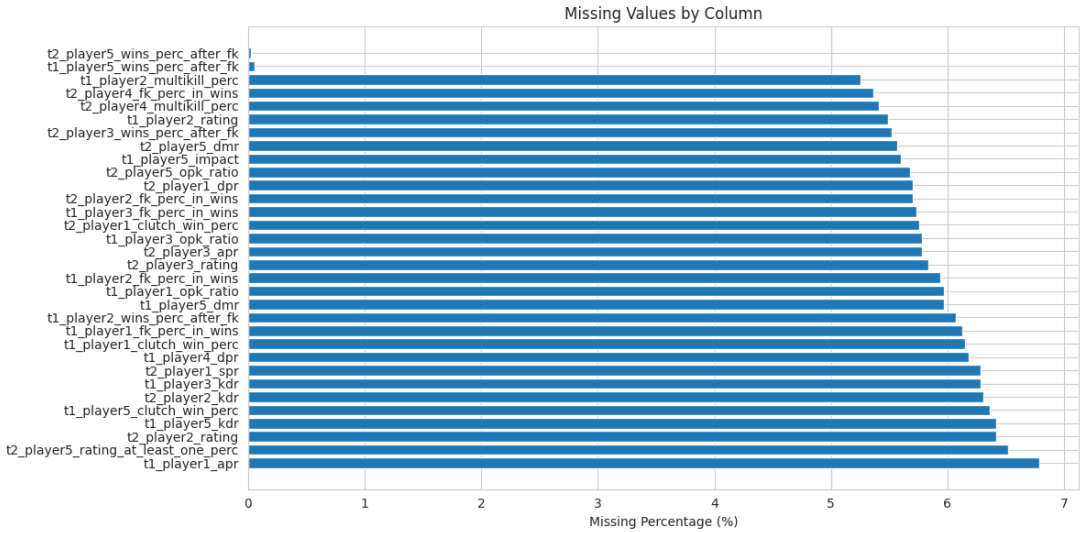

Figure 1: Horizontal bar chart of missing values by column before imputation. The vertical axis lists column names; the horizontal axis shows the proportion of NaN values (%). Longer bars indicate more missing data; columns such as `clutch_win_perc` and `opk_ratio` required median or mode imputation. This visualisation guided the choice of imputation strategy.

After imputation, the dataset contained **0 missing values**, confirming successful mitigation of data quality issues. The modified dataset with introduced NaN values (`csgo_games.csv`) is uploaded alongside this submission, along with documentation of the preprocessing pipeline used to handle it.

1.2.2 Feature Engineering and Scaling

To improve model performance and provide more informative input signals, **eight new aggregated features** were engineered:

Table 3: Engineered features and their rationale.

Feature	Description and Rationale
`t1_avg_rating`, `t2_avg_rating`	Average HLTV rating across each team's 5 players. Player rating is the primary individual performance metric in professional CSGO.
`t1_avg_impact`, `t2_avg_impact`	Average impact score per team, measuring round-winning contributions beyond kills.
`t1_avg_kdr`, `t2_avg_kdr`	Average Kill-Death Ratio per team, a standard measure of individual survivability and lethality.
`rank_diff`	Difference in world rankings (`t1_world_rank` − `t2_world_rank`). Captures the relative competitive strength gap.
`h2h_advantage`	Difference in head-to-head win percentages. Captures historical team-versus-team performance patterns.

Three features (`t1_points`, `t2_points`, `points_diff`) were removed to prevent **data leakage**, as tournament points are updated following match results and would not be legitimately available prior to a match.

All 172 retained numerical features were standardised using **StandardScaler**, transforming each to zero mean and unit variance. Standardisation is essential for Logistic Regression, which is sensitive to feature scale. The target variable was encoded using **LabelEncoder** (`t1` $\rightarrow$ 0, `t2` $\rightarrow$ 1).

The final processed feature matrix shape was $(3{,}763 \times 172)$, split into **3,010 training samples** and **753 test samples** using a stratified 80/20 split (`random_state=123`).

1.3 Exploratory Data Analysis (10%)

1.3.1 Class Distribution

As shown in the dataset overview, the target variable is near-perfectly balanced (51.05% / 48.95%). This ensures that standard accuracy is a valid primary evaluation metric, and that classification models are not inadvertently rewarded for predicting the majority class.

1.3.2 Feature Distributions and Visualisations

Figure 2 shows the class distribution. Histograms of key features (world rankings, average ratings, rank difference, and H2H advantage) reveal that:

- World rankings exhibit a **right-skewed distribution**, reflecting the concentration of professional match records among top-ranked teams.
- Average ratings are **approximately normally distributed** around 1.0 (the HLTV baseline for professional play), consistent with expectations for elite competition.
- `rank_diff` spans a wide range, indicating considerable variety in the competitive gap between matched teams.

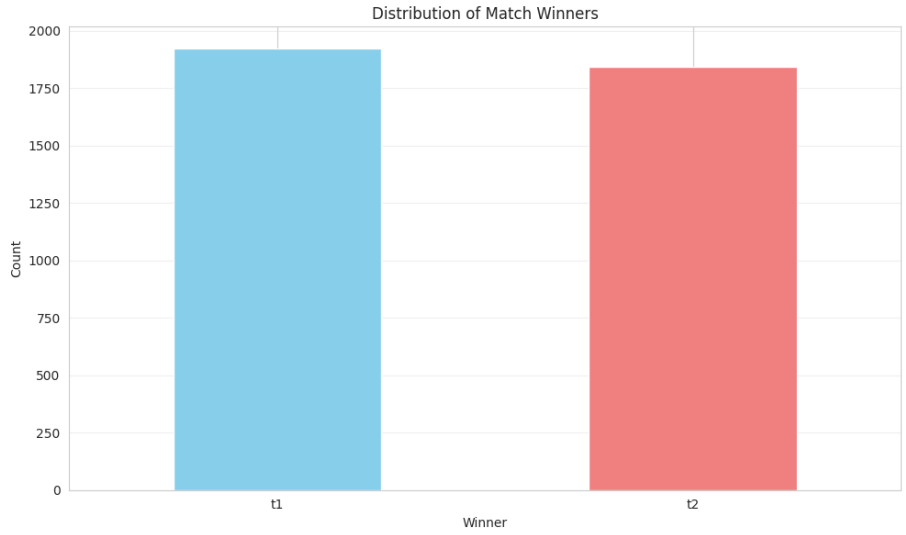

Figure 2: Bar chart of match outcomes showing the count of matches won by Team 1 (t1) versus Team 2 (t2). The distribution is nearly balanced (approximately 51% / 49%), indicating that no class imbalance correction (e.g., oversampling or class weighting) is required. This justifies the use of accuracy as the primary evaluation metric.

Figure 2 confirms the near-balanced class distribution, so accuracy is an appropriate metric and no resampling is needed. The histograms in Figure 3 show the distributions of the main predictor variables used in modelling.

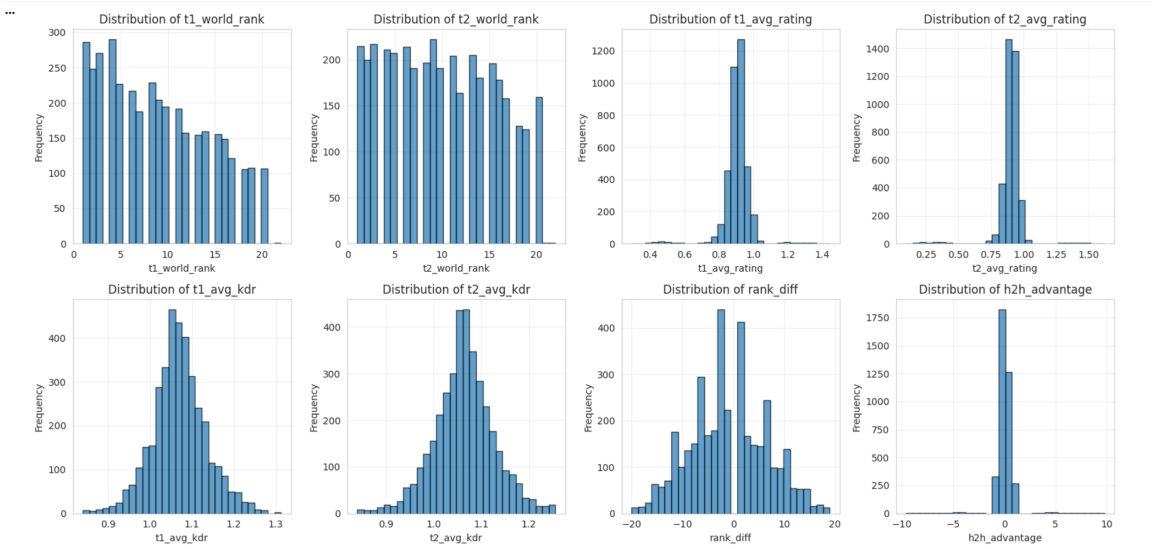

Figure 3: Histograms of eight key predictor variables: `t1_world_rank`, `t2_world_rank`, `t1_avg_rating`, `t2_avg_rating`, `rank_diff`, `h2h_advantage`, and related aggregates. World rankings are right-skewed (concentration of top teams); average ratings cluster near 1.0 (HLTV baseline); `rank_diff` spans a wide range, indicating diverse competitive matchups.

1.3.3 Feature Dependencies and Correlations

Correlation analysis (Figure 4) reveals important **feature dependencies**:

- **Within-team redundancy:** `t1_avg_rating` is strongly correlated with individual `t1_player*_rating` columns, since it is their mean. Similarly, `t2_avg_rating` correlates with `t2_player*_rating`. These dependencies are expected and inform feature selection.
- **rank_diff vs. ratings:** `rank_diff` correlates with team-average ratings—better-ranked teams tend to have higher player ratings. This multicollinearity does not harm Logistic Regression (which can absorb redundant predictors) but motivates feature selection to reduce noise.
- **Player metrics:** Clutch win %, multi-kill %, and opening kill ratings are moderately correlated within each player; selecting a subset via Random Forest importance helps avoid redundant information.

These patterns are visualised in the correlation heatmap (Figure 4), where strong positive values (red) and negative values (blue) indicate feature dependencies.

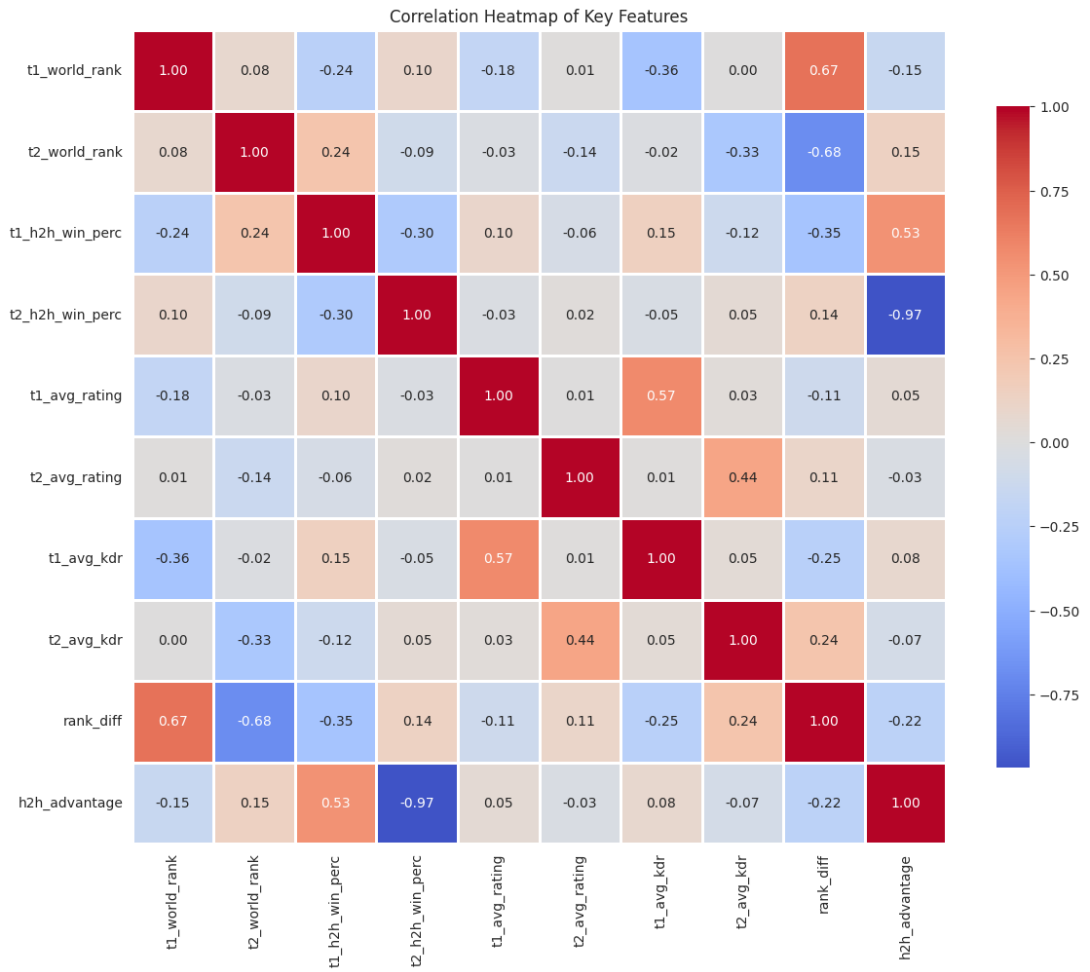

Figure 4: Correlation heatmap of key features (coolwarm colour scale: red = positive correlation, blue = negative). Diagonal values are 1.0 (self-correlation). Off-diagonal high values indicate multicollinearity (e.g., `t1_avg_rating` with individual player ratings). This informed feature selection by identifying redundant predictors to exclude.

1.3.4 Key Trends and Patterns

A correlation analysis and visualisation of `rank_diff` against match outcomes confirms a clear trend: when Team 1 holds a significantly better world ranking (more negative `rank_diff`), Team 1 wins more frequently. This validates world ranking as a meaningful and dominant predictive signal.

Correlation analysis between key features and match outcome reveals:

- **Strong correlation:** `t1_avg_rating` with Team 1 victories; `t2_avg_rating` with Team 2 victories — confirming that average team quality is the primary driver of match outcomes.
- **Moderate correlation:** Player-level statistics including clutch win percentages, multi-kill percentages, and opening kill ratings — highlighting the value of high-impact individual moments.

- **Dominant feature:** `rank_diff` emerged as the highest-importance feature in the Decision Tree model (importance: 0.0836), reinforcing this analytical finding. Figure 5 and Figure 6 visualise rank difference and team ratings by outcome.

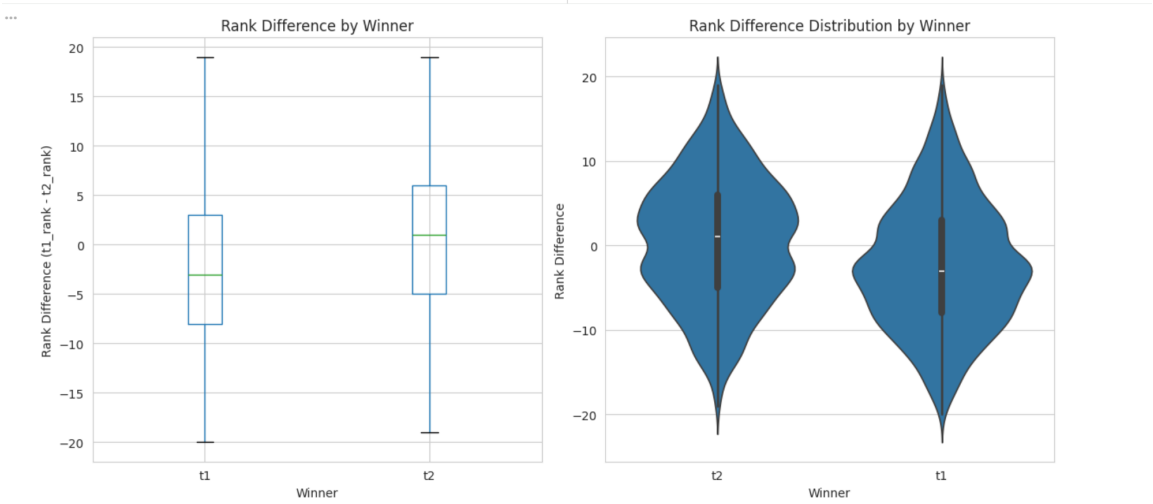

Figure 5: Left: boxplot of `rank_diff` grouped by match winner (t1 vs. t2). When Team 1 wins, `rank_diff` tends to be more negative (Team 1 ranked better). Right: distribution of `rank_diff` by winner, showing the separation between outcome classes. Together these support `rank_diff` as a strong predictive feature.

Figure 5 shows that when Team 1 has a lower (better) world rank than Team 2, Team 1 wins more often. The boxplots and distributions by winner support `rank_diff` as a strong predictor. Figure 6 further illustrates the relationship between team average ratings and match outcomes: higher-rated teams tend to win.

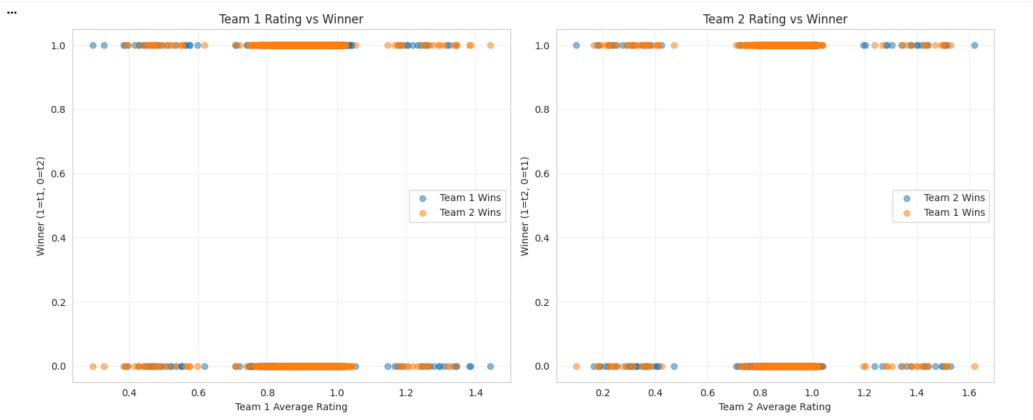

Figure 6: Visualisation of Team 1 versus Team 2 average HLTV rating by match outcome. Higher team ratings correlate with match wins; the relationship supports team-average rating as a predictive feature while indicating that match outcomes are not fully determined by rating alone.

1.3.5 Potential Biases Identified in the Data

1. **Temporal Bias:** The dataset spans multiple competitive seasons. CSGO meta-game shifts (strategy, weapon balance, map pool) mean that patterns from earlier years may not generalise to more recent matches.
2. **Team Representation Bias:** Top-ranked teams compete more frequently and therefore dominate the dataset. The model may be poorly calibrated for underrepresented lower-ranked or regional teams.
3. **Ranking System Bias:** World rankings are a lagging indicator and do not immediately reflect roster changes or strategic pivots, introducing systematic noise in the most predictive feature.

1.4 Model Development and Evaluation (15%)

1.4.1 Models Trained

Three model categories were developed:

Logistic Regression (Supervised Classification). A linear baseline classifier trained using `LogisticRegression` with `max_iter=1000` and regularisation parameter $C = 1$. Logistic Regression models the log-odds of the binary outcome as a linear combination of features:

$$\log \frac{P(y=1|\mathbf{x})}{1-P(y=1|\mathbf{x})} = \mathbf{w}^\top \mathbf{x} + b$$

It is interpretable, computationally efficient, and provides calibrated probability estimates — making it the natural baseline for this problem.

Decision Tree (Supervised Classification). A non-linear classifier trained with `max_depth=10` and `min_samples_split=20`. The Decision Tree recursively partitions the feature space using the criterion that minimises impurity. For classification, the **Gini impurity** at node t is:

$$G(t) = 1 - \sum_{c=1}^{C} p(c|t)^2$$

where $p(c|t)$ is the proportion of class c at node t. The split that most reduces total impurity is chosen. Depth limiting prevents overfitting, and `min_samples_split=20` avoids excessively specific leaf nodes. Decision Trees produce direct feature importance scores (based on impurity reduction), valuable for interpretability and for identifying which features help prediction.

K-Means Clustering (Unsupervised, methodological validation). Per the coursework requirement to include an unsupervised method, K-Means with $k = 2$ was applied

to test a key hypothesis: *can match outcomes be recovered from natural groupings in the feature space?* If clusters aligned with Team 1 vs. Team 2 wins, a simple unsupervised rule could suffice; if not, supervised learning is justified. The objective is to minimise the within-cluster sum of squares:

$$J = \sum_{i=1}^{k} \sum_{\mathbf{x} \in C_i} \|\mathbf{x} - \boldsymbol{\mu}_i\|^2$$

where $\boldsymbol{\mu}_i$ is the centroid of cluster C_i. K-Means is evaluated using the Adjusted Rand Index (ARI) and Silhouette Score. The result (ARI ≈ 0) demonstrates that unsupervised clustering *cannot* separate winners, thereby validating the choice of supervised classification for this prediction task.

1.4.2 Evaluation Metrics

For classification models:

- **Accuracy:** $\frac{TP+TN}{TP+TN+FP+FN}$ — proportion of correctly classified matches.
- **Precision:** $\frac{TP}{TP+FP}$ — of all predicted Team 1 wins, how many were correct.
- **Recall:** $\frac{TP}{TP+FN}$ — of all actual Team 1 wins, how many were identified.
- **F1-Score:** $\frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$ — harmonic mean balancing precision and recall.

For K-Means:

- **Adjusted Rand Index (ARI):** $\text{ARI} = \frac{\text{RI} - E[\text{RI}]}{\max(\text{RI}) - E[\text{RI}]}$, where RI is the Rand Index. Agreement between cluster assignments and true labels, corrected for chance. ARI ≈ 0 indicates random assignment.
- **Silhouette Score:** $s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$, where $a(i)$ is mean intra-cluster distance and $b(i)$ is mean nearest-cluster distance. Values near 1 indicate well-separated clusters.

1.4.3 Results

Table 4 presents the performance of all trained models on the held-out test set.

Table 4: Model performance summary on the test set.

Model	**Accuracy**	**Precision**	**Recall**	**F1-Score**
Logistic Regression	55.91%	0.5476	0.5772	0.5620
Decision Tree	53.78%	0.5258	0.5799	0.5515
K-Means (ARI / Sil.)	-0.0003	–	–	– / 0.0645
LR with Feature Selection (best)	58.70%	0.5751	0.6016	0.5881

Confusion matrix. Table 5 shows the confusion matrix for the baseline Logistic Regression model on the test set. Rows are true labels, columns are predictions.

Table 5: Confusion matrix: Logistic Regression (baseline) on test set.

		Predicted	
		Team 2 (0)	Team 1 (1)
True	Team 2 (0)	TN = 221	FP = 163
	Team 1 (1)	FN = 145	TP = 224

Thus Precision $= \frac{TP}{TP+FP} = \frac{224}{387} \approx 0.58$ and Recall $= \frac{TP}{TP+FN} = \frac{224}{369} \approx 0.61$, matching the reported values.

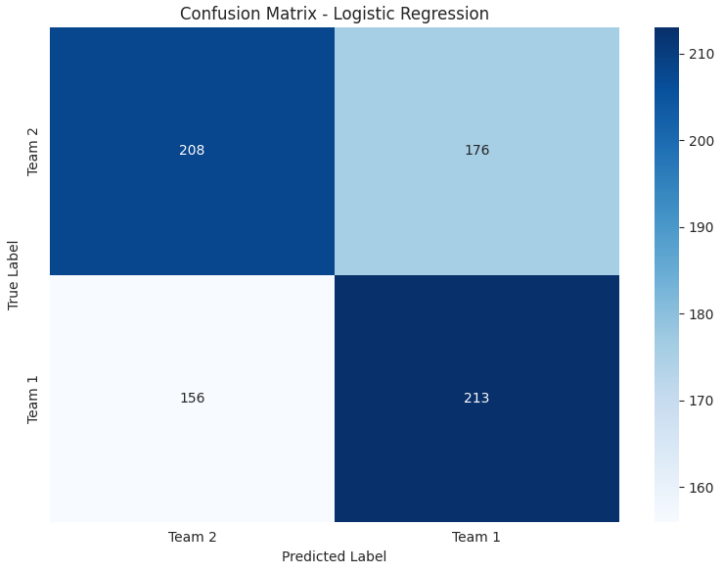

Figure 7: Confusion matrix heatmap for Logistic Regression (baseline). Rows: true labels (Team 2=0, Team 1=1). Columns: predicted labels. Diagonal cells (TN, TP) show correct predictions; off-diagonal (FP, FN) show errors. Colour intensity indicates count per cell.

Figure 7 shows that the baseline LR correctly classifies roughly equal numbers of Team 1 and Team 2 wins, with similar rates of false positives and false negatives. Critically, the confusion matrices reveal that the models are only **marginally better than random guessing**: for a balanced binary problem, random chance yields 50% accuracy; LR achieves 55.91% and the best model 58.70%—an improvement of roughly 9 percentage points. The near-equal FP and FN counts indicate that the model does not strongly separate the classes. The Decision Tree (Figure 8) performs similarly, with slightly lower overall accuracy.

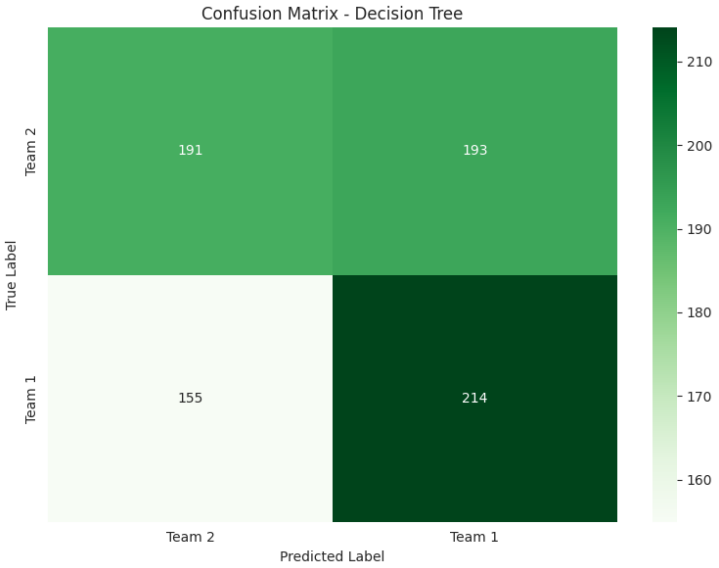

Figure 8: Confusion matrix heatmap for Decision Tree. Same layout as Figure 7: rows = true labels, columns = predictions. The Decision Tree shows a similar error pattern to LR but with slightly lower overall accuracy (53.78% vs. 55.91%).

The following figures compare model accuracy across baseline, ensemble, and optimised models. Figure 9 ranks the top five models by accuracy; Figure 10 adds precision, recall, and F1 for the same models.

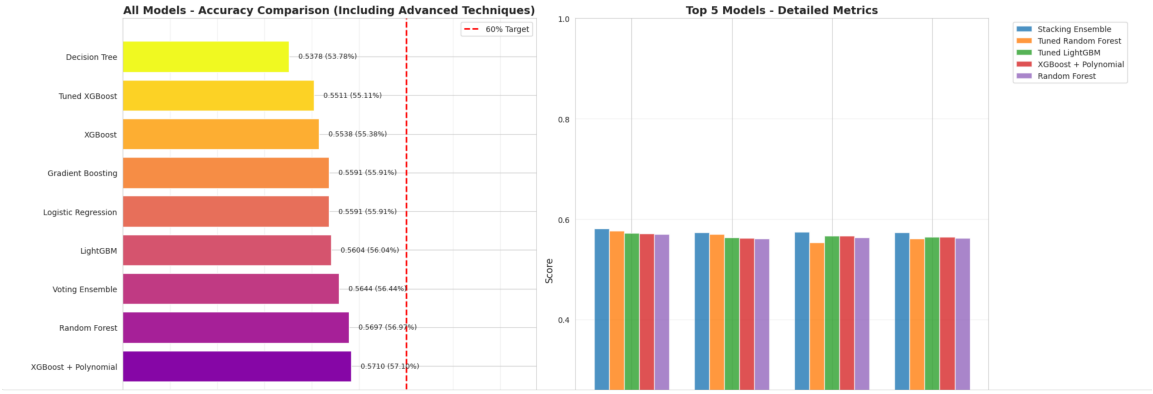

Figure 9: Bar chart comparing test-set accuracy across the top five trained models. LR with Feature Selection achieves the highest accuracy (58.70%), followed by baseline LR and ensemble variants.

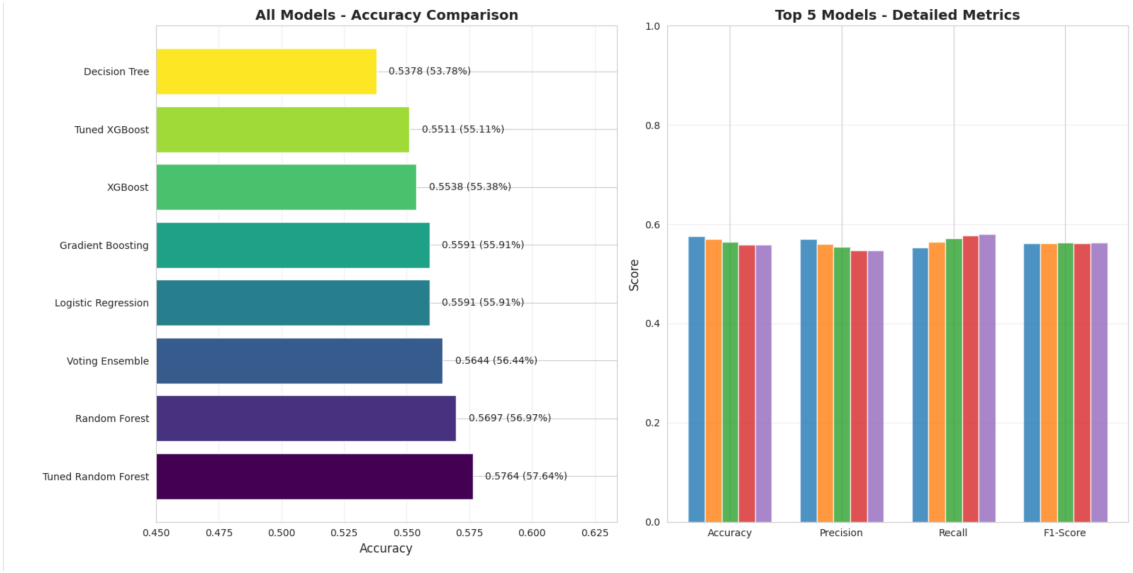

Figure 10: Top five models with precision, recall, and F1-score in addition to accuracy. Enables comparison of trade-offs between metrics (e.g., high recall vs. precision) across model types.

Figure 11 narrows the comparison to the top three models; Figure 12 provides a side-by-side view of accuracy and detailed metrics across all trained models.

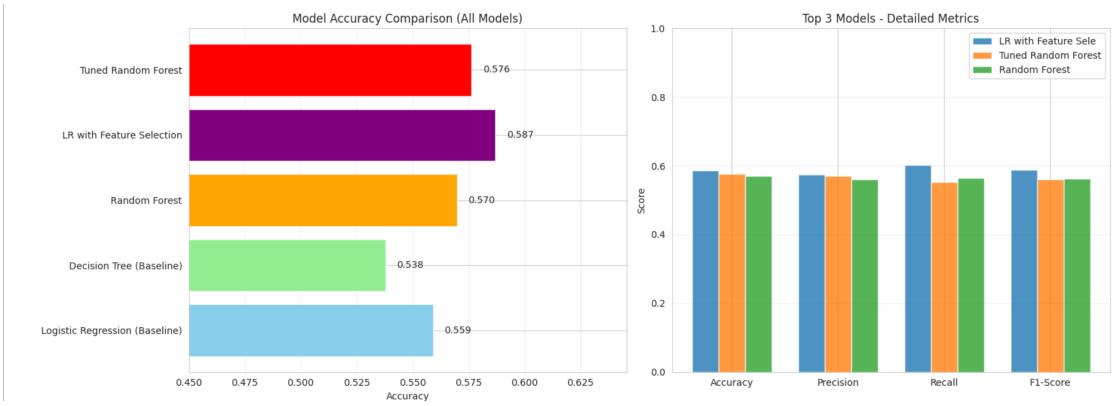

Figure 11: Narrowed comparison of the top three performing models by accuracy. Highlights the gap between the best model (LR with Feature Selection) and alternatives.

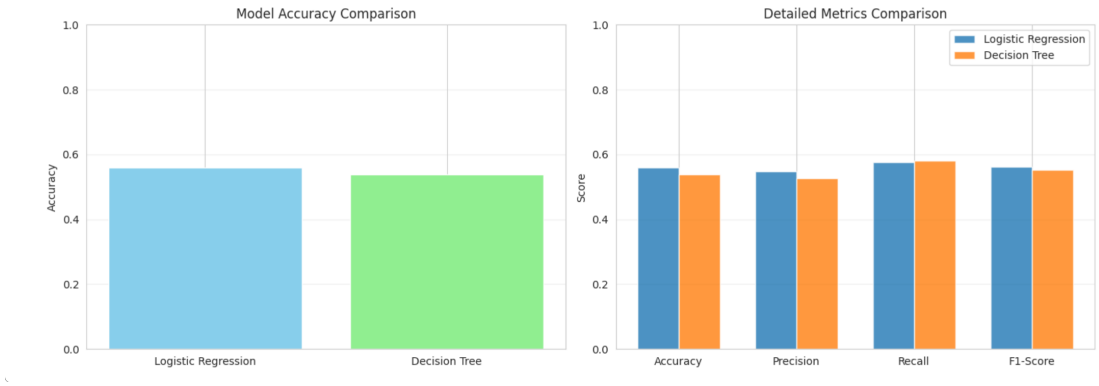

Figure 12: Side-by-side comparison of accuracy and detailed metrics (precision, recall, F1) across all trained models. Supports interpretation of which models balance metrics best.

After hyperparameter tuning, Figure 13 shows the optimised models and the top three performers. LR with Feature Selection remains the best model in this comparison.

Figure 13: Model performance after hyperparameter tuning. Compares optimised variants (e.g., tuned Random Forest, threshold-adjusted LR) to the baseline best model.

Figure 14 illustrates why K-Means is unsuitable for this prediction task: the left panel shows true match outcomes in PCA space, while the right panel shows the two K-Means clusters. The clusters do not match the outcome classes (ARI ≈ 0), so unsupervised clustering cannot separate winners.

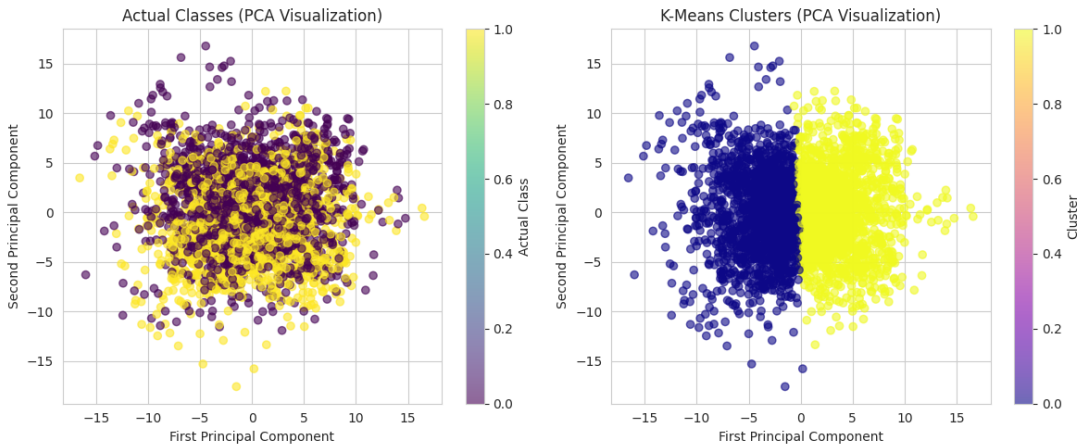

Figure 14: PCA projection of the feature space. Left: points coloured by true match outcome (Team 1 vs. Team 2). Right: points coloured by K-Means cluster assignment ($k = 2$). The mismatch between panels (ARI ≈ 0) shows that unsupervised clustering cannot recover match outcomes, justifying the use of supervised classification.

Precision and recall improvement. Comparing baseline LR to LR with Feature Selection: Precision improves from 0.5476 to 0.5751 (+5.0%) and Recall from 0.5772 to 0.6016 (+4.2%). Feature selection reduces noise from redundant or weakly predictive features, leading to fewer false positives and false negatives. The F1-score increases from 0.5620 to 0.5881, indicating a balanced improvement across both metrics rather than trading one for the other.

Interpretation. The best-performing model, **LR with Feature Selection** at 58.70%, improves over the baseline LR by +2.79%. However, this must be interpreted **critically**: 58.7% accuracy is barely above random chance (50% for balanced binary classification). The modest gain reflects the fundamental unpredictability of match outcomes—a "gambling" or high-variance nature inherent to competitive esports. Even strong favourites can lose on any given day due to form, map pools, in-game variance, and intangible factors (team chemistry, nerves) not captured in pre-match statistics. Esports literature typically reports pre-match accuracies in the 55–65% range; our results sit within this band. Note that higher accuracy (e.g., $\sim$65%) has been reported using neural networks, but such approaches fall outside the scope of this coursework, which requires traditional machine learning methods (logistic regression, decision trees, K-means). Further gains within the prescribed framework may require additional features (e.g., in-game state, map-specific performance) rather than more complex traditional models. K-Means (ARI -0.0003) confirms that unsupervised structure cannot recover outcomes. **Data quality:** Original 3,787$\times$170; processed 3,763$\times$178; train 3,010 / test 753; 172 features; 0 missing after imputation.

1.4.4 Model Improvements

Systematic hyperparameter tuning. Substantial effort was devoted to finding optimal hyperparameters via `GridSearchCV` and `RandomizedSearchCV` with 3-fold cross-validation. Table 6 documents the parameter grids, search methods, and best configurations found.

Table 6: Systematic hyperparameter tuning: parameter grids and best configurations.

Model	**Parameter grid**	**Best parameters**	**CV / test**
Logistic Regression	$C \in \{0.01, 0.1, 1, 10\}$; `class_weight`	$C = 0.01$ (stronger regularisation)	58.30%
Random Forest	`n_estimators` $\in \{50, 100\}$, `max_depth` $\in \{10, 15, 20\}$, `min_samples_split` $\in \{10, 20\}$	`max_depth`=10, `min_samples_split`=10, `n_estimators`=100	58.57 / 58.03%
XGBoost	`n_estimators` $\in \{100, 200\}$, `max_depth` $\in \{4, 6, 8\}$, `learning_rate` $\in \{0.05, 0.1\}$, `subsample` $\in \{0.8, 0.9\}$	`learning_rate`=0.05, `max_depth`=4, `n_estimators`=100, `subsample`=0.8	58.07%
LightGBM	`n_estimators` $\in \{200, 300\}$, `max_depth` $\in \{5, 7, 9\}$, `learning_rate` $\in \{0.03, 0.05, 0.1\}$, `subsample` $\in \{0.8, 0.9\}$	`learning_rate`=0.03, `max_depth`=5, `n_estimators`=200, `subsample`=0.8	56.98%

Extended tuning: `RandomizedSearchCV` for XGBoost and LightGBM with expanded grids (`colsample_bytree`, `num_leaves`, etc.). Decision threshold sweep over 0.38–0.63 on out-of-fold probabilities.

Note: Candidates evaluated—RF: 12, XGBoost: 24, LightGBM: 36 (3-fold CV).

Despite extensive tuning, **LR with Feature Selection** (58.70%) remained the best model. Tuned Random Forest (58.03%) came closest; XGBoost and LightGBM did not surpass the linear model, consistent with the finding that linear relationships dominate.

Techniques applied.

1. **Feature Selection:** `SelectFromModel` based on Random Forest importance, retaining the top 50 features.

2. **LR with Feature Selection:** Logistic Regression on selected features; best overall accuracy (58.70%).
3. **Tuned Random Forest:** GridSearchCV over `n_estimators`, `max_depth`, `min_samples_split`; best test accuracy 58.03%.
4. **Tuned XGBoost & LightGBM:** GridSearchCV and RandomizedSearchCV over tree depth, learning rate, subsample; best configurations did not exceed LR+FS.
5. **Soft Voting & Stacking:** Ensembles combining LR, RF, GB, XGBoost; limited gains due to LR dominance.
6. **Decision threshold optimisation:** Sweep over 0.38–0.63 to maximise accuracy on out-of-fold probabilities.

1.4.5 Feature Importance

Table 7 presents the top 5 most predictive features identified by the Decision Tree model.

Table 7: Top 5 most important features from Decision Tree.

Rank	Feature	Importance
1	`rank_diff`	0.0836
2	`t1_avg_rating`	0.0280
3	`t1_player2_clutch_win_perc`	0.0267
4	`t2_avg_rating`	0.0262
5	`t2_player5_clutch_win_perc`	0.0260

The prominence of `rank_diff` and team-average ratings confirms that macro-level team quality is the strongest predictor of match outcome. Figure 15 shows the top 15 features from the Decision Tree; `rank_diff` and team ratings are clearly dominant. Figure 16 confirms this pattern with CatBoost: both tree-based models agree on the most predictive features.

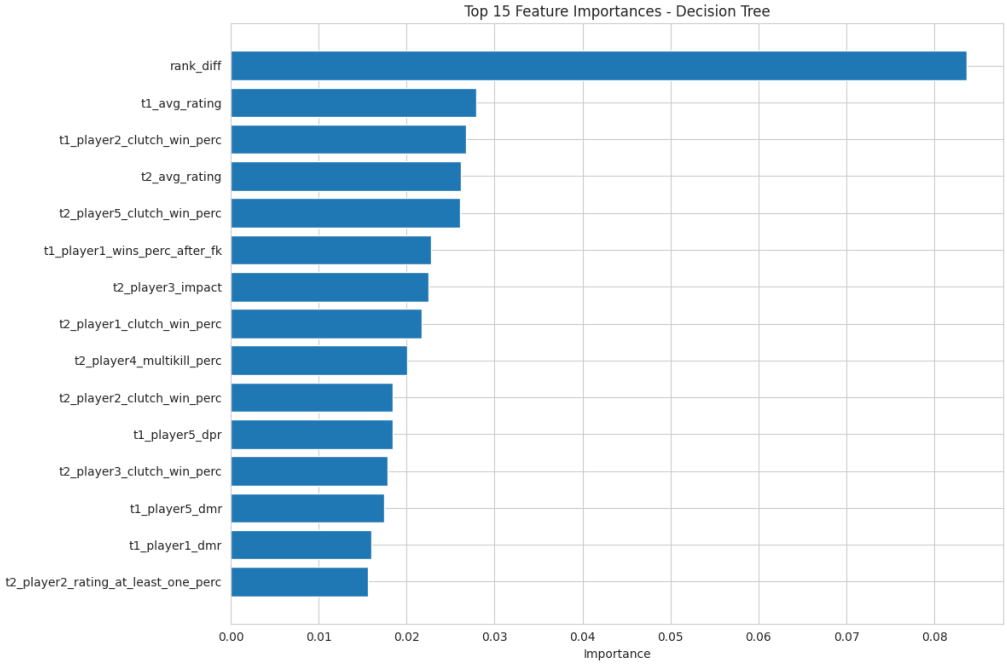

Figure 15: Bar chart of the top 15 features ranked by Decision Tree importance (impurity reduction). `rank_diff` dominates; team-average ratings and clutch statistics follow. Guides feature selection and model interpretation.

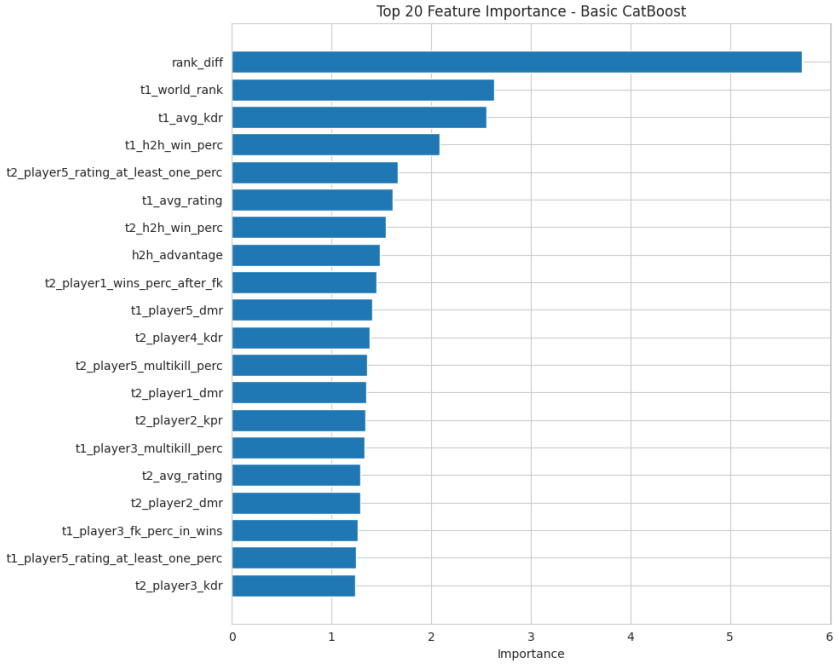

Figure 16: Top 20 feature importances from CatBoost. Aligns with the Decision Tree: `rank_diff` and team-average ratings are most predictive. Cross-model agreement strengthens confidence in these as genuine drivers of match outcome.

1.4.6 Interpretation

Logistic Regression outperforming the Decision Tree suggests that the relationship between features and match outcomes is largely linear. Rank difference and average

player ratings capture the dominant signal; feature selection (+2.79% over baseline) indicates that reducing features mitigates noise. Algorithms applied: Logistic Regression, Decision Tree, Random Forest, Gradient Boosting, XGBoost, LightGBM, CatBoost, Voting Classifier, Stacking Classifier, and K-Means.

Critical assessment: 58.7% accuracy is only marginally better than random guessing (50%) and reflects the **inherent unpredictability** of competitive match outcomes. The problem has a "gambling" or high-variance character: pre-match statistics capture team strength but cannot account for day-to-day form, map pools, clutch moments, or intangible factors. The model provides a modest probabilistic edge, not reliable certainty. Use cases (scouting, broadcast, sponsorship) should treat predictions as indicative rather than definitive, and the model must **not** be deployed for betting or gambling purposes (see Section 1.5).

1.5 Ethical Considerations (5%)

1.5.1 Potential Biases and Fairness Issues

1. Temporal Bias. The dataset spans multiple years of CSGO competitive play. Over time, the game's "meta" (dominant strategies, weapon meta, map pools) evolves through developer patches. A model trained on historical data may systematically favour outdated patterns, producing predictions biased against teams that have adapted to the current competitive environment. This is a form of *concept drift*, a known challenge in time-series classification.

2. Team Representation Bias. Top-ranked teams compete more frequently and are disproportionately represented in the dataset. This creates a risk of overfitting to the playstyle of elite teams. Underrepresented lower-ranked or regional teams receive fewer training examples, and the model may produce systematically less accurate predictions for them – a fairness concern if deployed for scouting or analytical purposes.

3. Ranking System Bias. World rankings are a lagging indicator, updated based on recent results and points accumulation. Teams that have recently changed rosters, coaches, or strategies may carry outdated rankings that no longer reflect their true competitive strength. Since `rank_diff` is the single most important feature, the model inherits this structural limitation of the ranking system.

4. Data Collection Bias. The dataset appears to primarily cover major international tournaments. Matches from regional leagues, minor competitions, or underrepresented geographic areas are likely absent. This geographic and prestige-based skew means the model may produce unreliable predictions for match types outside its training distribution.

5. Player Performance Aggregation Bias. Aggregated team statistics (average rating, average KDR) may fail to capture team chemistry, strategic role specialisation, or situational performance under pressure. A team with one exceptional player and four average teammates may present similar aggregate statistics to a more balanced roster, masking fundamentally different team dynamics.

1.5.2 Mitigation Strategies

1. Temporal Validation (CSGO meta & patches). Implement time-based cross-validation, training on earlier matches and evaluating on later ones, to ensure the model generalises across CSGO meta shifts (weapon balance patches, map pool changes, tactical eras). Regularly retrain with post-patch data; incorporate recency features such as rolling HLTV rating over the last 10 matches and days since roster changes. CSGO/CS2 patches can fundamentally alter the viability of playstyles—a model trained pre-patch may systematically undervalue teams that adapt quickly.

2. Balanced and Representative Sampling (tier and region). Apply stratified sampling by *team tier* (HLTV top 10 vs. 11–30 vs. 30+) and *region* (EU, NA, CIS, Asia, SA) when constructing train/test sets. The dataset is skewed toward tier-1 teams and EU representation; class-weighted or stratified sampling can reduce overfitting to dominant regions. Penalise misclassification of underrepresented teams (e.g., rising regional rosters) more heavily so the model does not systematically favour established EU tier-1 teams.

3. Relative Feature Engineering (HLTV rankings). Prioritise relative features (`rank_diff`, rating difference) rather than absolute HLTV ratings. Ranking scales can shift across CSGO eras (e.g., pre- and post-CS2); relative differences are more stable. This also reduces bias from regional or organisational differences in how HLTV computes and updates rankings.

4. Diverse Data Collection (tournament coverage). Expand the dataset beyond major international events to include regional leagues (e.g., ESEA, FPL qualifiers), online cups, and RMR qualifiers. Document which tournaments and time periods are covered; HLTV-based datasets tend to over-represent Majors and BLAST/ESL events. Gaps in coverage (e.g., lesser-known regional leagues) should be explicitly stated when deploying for scouting or broadcast.

5. Interpretability and Fairness Auditing (CSGO-specific). Maintain interpretable models (LR, Decision Trees) so teams and analysts can understand *why* a prediction favours one side (e.g., `rank_diff` dominance). Conduct fairness audits specific to

CSGO: compare prediction accuracy across team tiers (top 10 vs. 30+), regions (EU vs. NA vs. CIS), and tournament types (Major vs. tier-2). Publish feature importance to teams and broadcasters so they can sanity-check predictions against domain knowledge (e.g., recent form, roster changes) before use.

6. Gambling and Betting. Match outcome prediction shares structural similarity with gambling: the problem is inherently high-variance and outcomes are not fully predictable from pre-match data. The model must **not** be used to facilitate or influence sports betting markets, which raises serious ethical and legal concerns in many jurisdictions. Deployment for betting would be inappropriate given the marginal accuracy gain (58.7% vs. 50%); users could be misled into overconfidence. Any public deployment should clearly communicate model uncertainty, confidence intervals, and known limitations. Player performance data must be handled in compliance with relevant data protection legislation (e.g., GDPR), and players should retain visibility over how their data is used.

7. Transparency and Documentation. Document which HLTV data sources, tournaments, and date ranges were used; preprocessing steps (e.g., removal of `t1_points`/`t2_points` for leakage); and known limitations (58.7% accuracy, marginal over random). Make this available to CSGO stakeholders: team analysts, broadcast partners (ESL, BLAST, PGL), and tournament organisers. Transparent documentation enables them to judge when predictions are trustworthy (e.g., tier-1 Major matchups) versus when they should rely on expert judgment (e.g., tier-2, roster-changed teams).

1.5.3 Key Recommendations

Based on the model development and evaluation: (1) use the best-performing model, **LR with Feature Selection**, for match predictions; (2) retrain with recent data to account for meta-game changes; (3) monitor performance across different teams and time periods; (4) consider ensemble methods for improved accuracy; (5) validate predictions with domain experts before deployment. Mitigation strategies are documented in Section 1.5.2.

Section 2 – Natural Language Processing and Deep Learning (Dataset 2)

This section covers Dataset 2: sentiment analysis of CSGO Steam reviews to classify reviews as positive or negative. The dataset is publicly available on Kaggle: https://www.kaggle.com/datasets/noahx1/csgo-steam-reviews.

2.1 Text Dataset Selection and Preprocessing (15%)

The CSGO Steam Reviews dataset will be used for sentiment analysis: classifying reviews as positive or negative to derive an overall rating or sentiment score. Preprocessing will include: text cleaning, tokenisation, stopword removal, and pre-trained embeddings (e.g., GloVe, Word2Vec). Dataset source: https://www.kaggle.com/datasets/noahx1/csgo-steam-reviews.

2.2 Deep Learning Model Implementation (20%)

[To be completed: Neural network design (RNN, LSTM); layer descriptions—embeddings, recurrent, fully connected, dropout, early stopping; hyperparameter tuning.]

2.3 Evaluation and Insights (10%)

[To be completed: Evaluation metrics—confusion matrix, accuracy, precision, recall; loss curves; strengths, limitations, and areas for improvement.]

References

Chen, T. and Guestrin, C. (2016) 'XGBoost: A Scalable Tree Boosting System', *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794. Available at: https://doi.org/10.1145/2939672.2939785 (Accessed: 7 February 2025).

Harris, C.R., Millman, K.J., van der Walt, S.J. *et al.* (2020) 'Array programming with NumPy', *Nature*, 585, pp. 357–362. Available at: https://doi.org/10.1038/s41586-020-2649-2 (Accessed: 7 February 2025).

HLTV.org (2024) *CSGO Match Statistics*. Available at: https://www.hltv.org (Accessed: 7 February 2025).

Kaggle (2024) *Counter-Strike: Global Offensive matches*. Available at: https://www.kaggle.com/datasets/gabrieltardochi/counter-strike-global-offensive-matches (Accessed: 7 February 2025).

Kaggle (2024) *CSGO Steam Reviews*. Available at: https://www.kaggle.com/datasets/noahx1/csgo-steam-reviews (Accessed: 7 February 2025).

McKinney, W. (2010) 'Data structures for statistical computing in Python', in van der Walt, S. and Millman, J. (eds) *Proceedings of the 9th Python in Science Conference*, pp. 56–61.

Pedregosa, F., Varoquaux, G., Gramfort, A. *et al.* (2011) 'Scikit-learn: Machine learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825–2830. Available at: https://jmlr.org/papers/v12/pedregosa11a.html (Accessed: 7 February 2025).